S&DS 365 / 665

Intermediate Machine Learning

Reinforcement Learning: Policy Gradient Methods

(Continued)

November 5

Yale

Version of 2025/11/05 at 11:31

Reminders

- Assignment 4 due Thursday, November 20
- Quiz 4 posted today on Canvas at 2:30pm (48 hours/30 min)

Reminders

- Assignment 4 due Thursday, November 20
- Quiz 4 posted today on Canvas at 2:30pm (48 hours/30 min)
  - Graphs
  - GNNs
  - Q-learning
  - Midterm

Reminders

- Assignment 4 due Thursday, November 20
- Quiz 4 posted today on Canvas at 2:30pm (48 hours/30 min)
  - Graphs
  - GNNs
  - Q-learning
  - Midterm
- Final exam: Friday, December 12, 2025 at 9am
  - https://catalog.yale.edu/ycps/final-examination-schedules/

Outline for today

- Policy gradient methods (continued)
- Combining policy and value estimation
- Demo: Cartpole

Main reference: “Reinforcement Learning: An Introduction” (Second Edition), Richard S. Sutton and Andrew G. Barto

Policy gradient methods: Loss function

We start with the loss function: Expected reward $\mathcal{J}(\theta) = \mathbb{E}(R)$

- Parameterize the policy—$\pi_\theta(a \mid s)$—and extract "features" of states
- Perform gradient ascent of $\mathcal{J}(\theta)$
- Well-suited to deep learning approaches

Policy gradient methods: Loss function

Policy is probability distribution $\pi_\theta(a \mid s)$ over actions given state s.

The episode unfolds as a random sequence—a "rollout" of the current policy:

$$\tau : (s_0, a_0) \to (s_1, r_1, a_1) \to (s_2, r_2, a_2) \to \cdots \to (s_T, r_T, a_T) \to s_{T+1}$$

where s_{T+1} is a terminal state.

Total reward $R(\tau) = \sum_{t=1}^{T} r_t$

Expected reward $\mathcal{J}(\theta) = \mathbb{E}_\theta(R(\tau))$

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\mu(\tau)$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

Calculating the gradient

Using Markov property, calculate $\mathbb{E}_\theta(R(\tau))$ as

$$\mathbb{E}_\theta(R(\tau)) = \int p(\tau \mid \theta) R(\tau)\, d\mu(\tau)$$

$$p(\tau \mid \theta) = \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t)$$

If states or rewards are finite the integral becomes a sum, or a mix of sums and integrals.

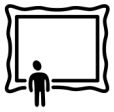
Two factors contribute to probability of a path τ:

- Agent action probabilities — dependent on θ
- Environment state/reward transitions — not dependent on θ

The key equation

We showed via the "grad-log trick" and $\log(\pi_\theta \cdot p) = \log \pi_\theta + \log p$ that

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_\theta \left(R(\tau) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right)$$



Why is this important?

- This manipulation is important because it gets the policy explicity into the objective function as an expectation
- Similar to the “reparameterization trick” for VAEs

Approximating the gradient

We next approximate this using derivatives over N "rolled out" paths:

$$\widehat{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{i=1}^{N} R(\tau^{(i)}) \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})$$

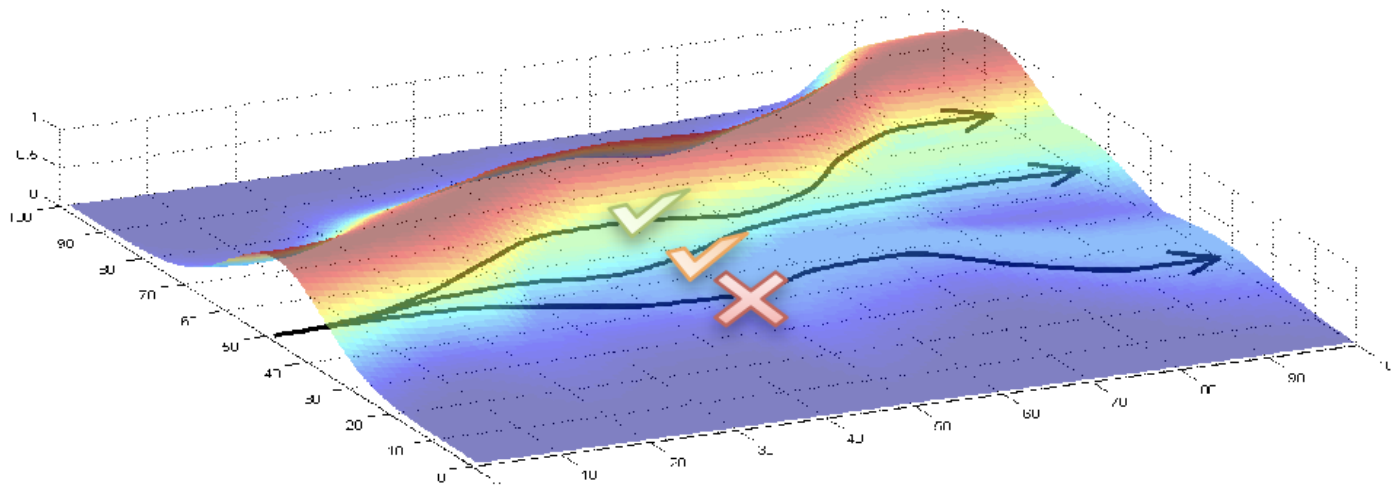
The policy parameters are then updated as

$$\theta \longleftarrow \theta + \alpha \widehat{\nabla_\theta \mathcal{J}(\theta)}$$

Recall: Intuition

Gradient ascent of the expected reward with respect to policy:

- Increases probability of paths with large R
- Decreases probability of paths with small R



How to make this more efficient?

- If rewards are all positive, we might increase weight on all actions — with larger increases when rewards are greater
- We actually want to boost up actions associated with rewards that have *greater than average* reward
- Similarly, downweight actions associated with rewards that have below average reward
- This can *reduce the variance* of policy estimates

Baseline subtraction

Our “rollout” gradient

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) R(\tau^{(i)})$$

is an unbiased estimate of the true model gradient:

$$\mathbb{E}\left(\frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) R(\tau^{(i)}) \right) = \nabla_\theta \mathcal{J}(\theta)$$

Baseline subtraction

But then

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta) \left(R(\tau^{(i)}) - b \right)$$

is *also* an unbiased estimate of the gradient, where b is a "baseline" average.

Why?

Baseline subtraction

This is because

$$\begin{aligned}
\mathbb{E}\left(\nabla_\theta \log p(\tau \mid \theta)\, b\right) &= \sum_\tau p(\tau \mid \theta) \nabla_\theta \log p(\tau \mid \theta)\, b \\
&= \sum_\tau \nabla_\theta p(\tau \mid \theta)\, b \\
&= \nabla_\theta \sum_\tau p(\tau \mid \theta)\, b \\
&= b \nabla_\theta \sum_\tau p(\tau \mid \theta) \\
&= b \nabla_\theta 1 \\
&= 0
\end{aligned}$$

Same result will hold as long as b doesn't depend on the actions a_t

Introducing time dependence

Now we can refine this. The gradient is

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \log p(\tau^{(i)} \mid \theta)(R(\tau^{(i)}) - b)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \right) \left(\sum_{t=0}^{T} r_t^{(i)} - b \right)$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=0}^{t} r_k^{(i)} + \sum_{k=t+1}^{T} r_k^{(i)} - b \right) \right)$$

But action a_t only impacts rewards after time t.

Here we're using the reward at step k as $r_k = \text{reward}(s_k, a_k)$

Introducing time dependence

So, we replace this with

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b \right)$$

or

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b(s_t^{(i)}) \right)$$

We weight the gradients at time t by the total reward from that time onwards minus the average reward

This makes more efficient use of the samples.

Estimating b

We use the improved (lower variance) gradient estimate

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) \left(\sum_{k=t+1}^{T} r_k^{(i)} - b(s_t^{(i)}) \right)$$

Estimate b with regression using another neural network with parameters ϕ:

$$\phi \leftarrow \arg\min_{\phi} \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \left(\sum_{k=t+1}^{T} r_k^{(i)} - b_\phi(s_t^{(i)}) \right)^2$$

Using the value function

But the average discounted reward starting from the state s_t is the *value function*:

$$b(s_t) = \mathbb{E}(r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{T-t} r_T) = V^\pi(s_t)$$

So, we are led to simultaneously look for an estimate of the value function while estimating the policy.

This is a type of RL called *actor-critic reinforcement learning*.

Using the Bellman equation

- Roll out policy, collecting paths $\tau^{(i)}$ and instances of (s, a, s_{next}, r)
- Update value function:

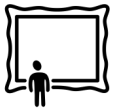
$$\phi \leftarrow \arg\min_{\phi} \sum_{(s,a,s_{next},r)} \left(r + \gamma V^{\pi}_{\phi_{old}}(s_{next}) - V^{\pi}_{\phi}(s) \right)^2 + \lambda \|\phi - \phi_{old}\|^2$$

- Update policy:

$$\theta \leftarrow \theta + \eta \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_{\theta} \log \pi(a_t^{(i)} \mid s_t^{(i)}) \underbrace{\left(\sum_{k=t+1}^{T} \gamma^{k-t-1} r_k^{(i)} - V^{\pi}_{\phi}(s_t^{(i)}) \right)}_{\textit{Advantage}}$$

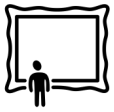
Summary so far

- Policy gradients have a nice form
- Can reduce variance in gradient ascent by comparing to baseline
- Natural baseline is given by the value function
- Value function can be estimated during policy estimation
- Modern RL methods develop different refinements to this approach



Actor-critic approaches: Idea

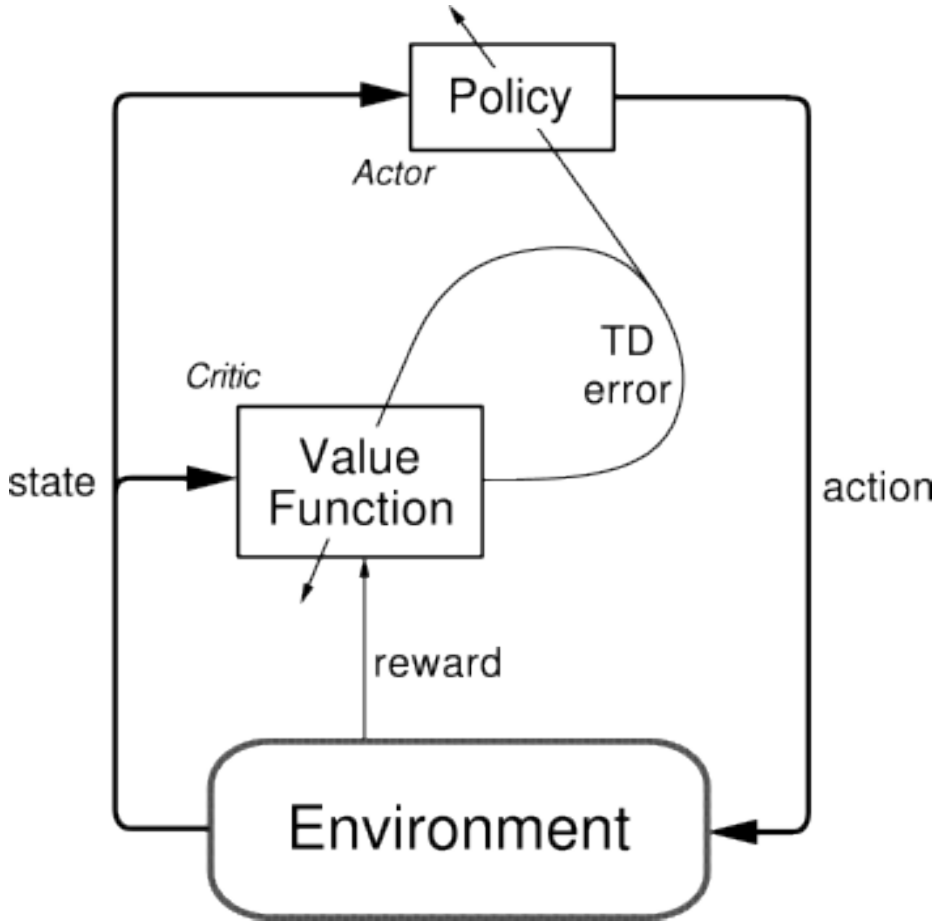
- Estimate policy and value function together
- Actor: policy used to select actions
- Critic: value function used to criticize actor
- Error signal from the critic drives all learning



Actor-critic approaches

- After each selected action, critic evaluates new state
- Have things gone better or worse than expected?
- The error signal is used to update actor and value function

Actor-critic approaches



Sutton

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

$\delta_t > 0$: action was better than expected

$\delta_t < 0$: action was worse than expected

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Value function is updated as

$$V(s_t) \leftarrow V(s_t) + \alpha\delta_t$$

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Used to update parameters of policy.

If δ_t is positive (negative), action a_t should become more (less) probable in state s_t

For example, with

$$\pi_\theta(a \mid s) = \text{Softmax}\Big\{ f_\theta(s, a_1), \ldots, f_\theta(s, a_D) \Big\}$$

parameters θ adjusted so $f_\theta(s_t, a_t)$ increases (decreases)

Actor-critic approaches

Error signal is

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

Used to update parameters of policy.

For example, with

$$\pi_\theta(a \mid s) = \text{Softmax}\Big\{ f_\theta(s, a_1), \ldots, f_\theta(s, a_D) \Big\}$$

parameters θ adjusted so $f_\theta(s_t, a_t)$ increases (decreases)

$$\theta \leftarrow \theta + \eta \nabla_\theta \log \pi_\theta(a_t \mid s_t) \Big(r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \Big)$$

$$= \theta + \eta \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \delta_t$$

Returning to our policy gradient approach

- Roll out policy, collecting paths $\tau^{(i)}$ and instances of (s, a, s_{next}, r)
- Update value function:

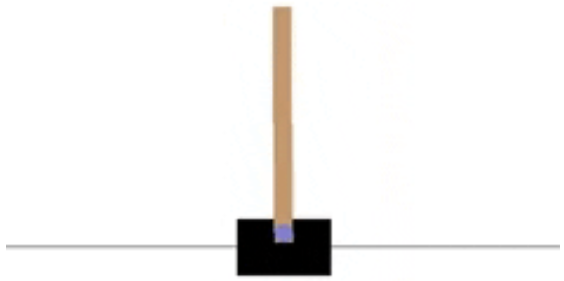
$$\phi \leftarrow \arg\min_{\phi} \sum_{(s,a,s_{next},r)} \left(r + \gamma V^{\pi}_{\phi_{old}}(s_{next}) - V^{\pi}_{\phi}(s) \right)^2 + \lambda \|\phi - \phi_{old}\|^2$$

- Update policy:

$$\theta \leftarrow \theta + \eta \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_{\theta} \log \pi(a_t^{(i)} \mid s_t^{(i)}) \underbrace{\left(r_{t+1} + \gamma V^{\pi}_{\phi}(s_{t+1}^{(i)}) - V^{\pi}_{\phi}(s_t^{(i)}) \right)}_{\textit{Advantage}}$$

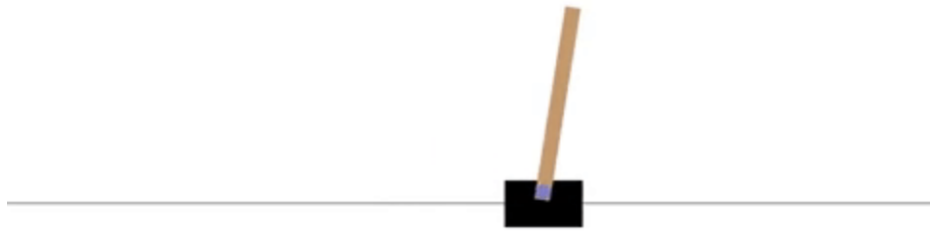
CartPole-v0

A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The system is controlled by applying a force of +1 or -1 to the cart. The pendulum starts upright, and the goal is to prevent it from falling over. A reward of +1 is provided for every timestep that the pole remains upright. The episode ends when the pole is more than 15 degrees from vertical, or the cart moves more than 2.4 units from the center.



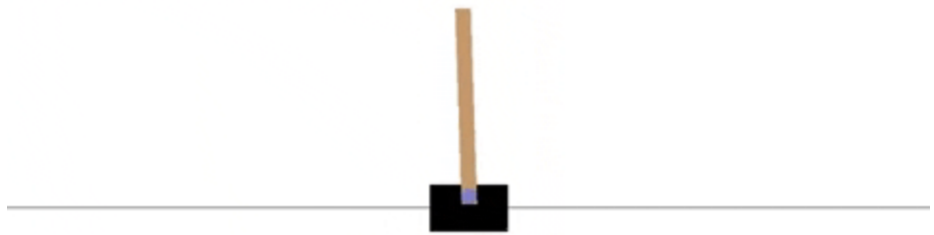
https://gym.openai.com/envs/CartPole-v0/
tensorflow tutorial

Before learning:



https://gym.openai.com/envs/CartPole-v0/
tensorflow tutorial

After learning:



https://gym.openai.com/envs/CartPole-v0/
tensorflow tutorial

Demo

As you study the code, you will see how

1. Actor log-probs, critic values, and rewards are buffered
2. After the episode, TD errors are calculated at each time step
3. Loss for actor is negative log-prob weighted by TD error
4. Loss for critic is absolute value of TD error
5. Gradients of actor/critic networks computed using auto diff
6. Parameters of network are updated

Summary: Actor-critic RL

- Estimate policy and value function together
- Actor: policy used to select actions
- Critic: value function used to criticize actor
- Error signal from the critic is deviation from Bellman equation
- This drives learning — violation of expected rewards